Reporte

Nombre: Equipo 6 Luis David Cigala Montes Jared Alejandro Garcia Alcocer Gabriel Sánchez Rivera	**Matrícula**: 2120993 2119041 2121131
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Actividad 4, Diagrama de estados y Domain Events (TechStore)
Fecha: 10/28/26	
Bibliografia (Formato APA): Archivo de la actividad 4 brindada por el ingeniero	

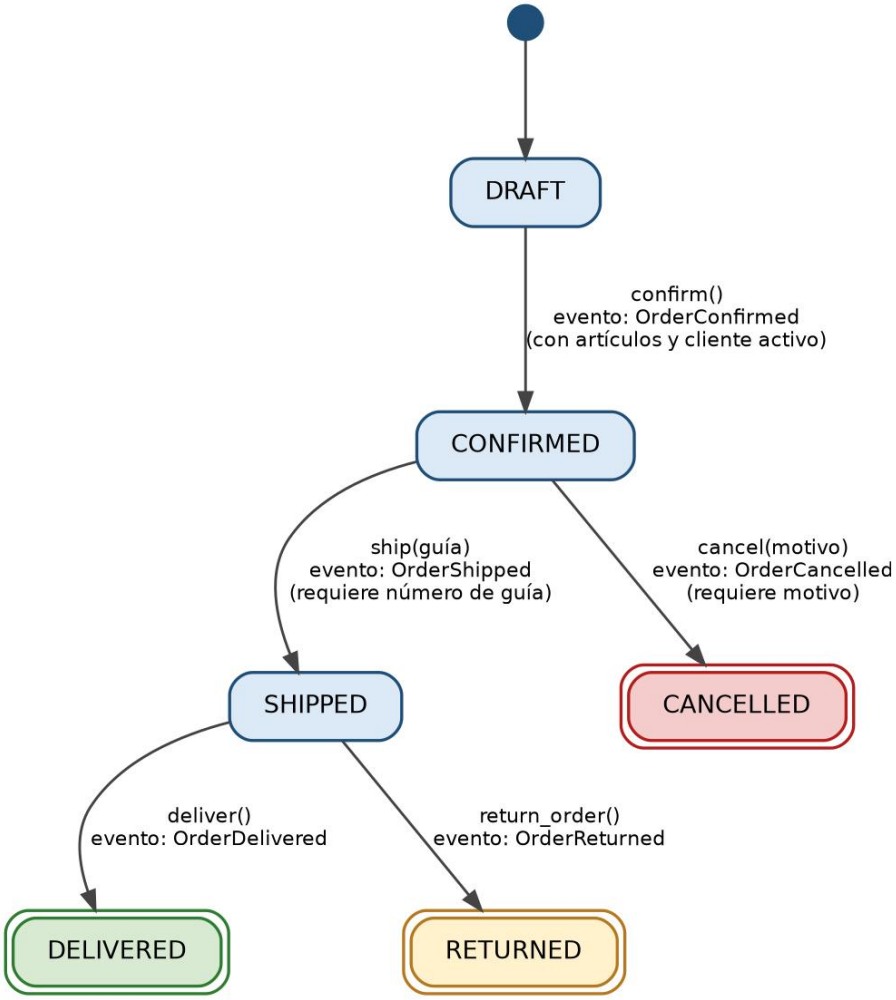
Objetivo:

Representar con un diagrama de estados el ciclo de vida del pedido de TechStore y explicar cuándo se registra cada Domain Event, según el código de la actividad 4.

Procedimiento:

1. Diagrama de estados del pedido

El pedido (Order) empieza en DRAFT y cambia de estado solo mediante sus métodos. Los estados con doble borde (DELIVERED, CANCELLED y RETURNED) son finales: de ellos no sale ninguna transición.



2. Transiciones permitidas

Método	Transición	Condiciones	Evento que registra	Si no se cumple
confirm()	DRAFT a CONFIRMED	Al menos un artículo y cliente activo	OrderConfirmed	Si no está en DRAFT: InvalidOrderTransition (HTTP 409). Sin artículos o con cliente inactivo: InvalidOrderData (HTTP 422)

ship(guía)	CONFIRMED a SHIPPED	Número de guía no vacío	OrderShipped	Si no está en CONFIRMED: 409. Sin guía: 422
cancel(motivo)	CONFIRMED a CANCELLED	Motivo no vacío	OrderCancelled	Si no está en CONFIRMED (por ejemplo SHIPPED): 409. Sin motivo: 422
deliver()	SHIPPED a DELIVERED	Ninguna adicional	OrderDelivered	Si no está en SHIPPED: 409
return_order()	SHIPPED a RETURNED	Ninguna adicional	OrderReturned	Si no está en SHIPPED: 409

Cualquier salto que no aparezca en el diagrama se rechaza. Además, add_item solo permite agregar artículos mientras el pedido está en DRAFT, y el pedido no puede superar las 20 unidades.

3. Domain Events

Un Domain Event es un objeto inmutable (DomainEvent, con frozen=True) que comunica que un hecho del negocio ya ocurrió. Guarda tres datos: el nombre del evento, el id del pedido y la fecha y hora en UTC. Cada pedido acumula sus eventos en la lista order.events.

Evento	**Cuándo se registra**	**Qué comunica**
OrderConfirmed	Al terminar confirm()	El pedido fue aceptado por la tienda
OrderShipped	Al terminar ship()	El pedido salió con su número de guía
OrderCancelled	Al terminar cancel()	El pedido fue cancelado con un motivo
OrderDelivered	Al terminar deliver()	El cliente recibió el pedido

OrderReturned	Al terminar return_order()	El pedido enviado fue devuelto

Regla principal: el evento se registra solo después de que pasaron todas las validaciones y de que el estado cambió. En cada método, la línea que agrega el evento está al final; si algo falla antes, se lanza la excepción y esa línea nunca se ejecuta. Por eso una operación inválida no cambia el estado ni añade eventos.

Ejemplo comprobado con el pedido O001: al ejecutar confirm(), ship("G123") y deliver(), el pedido termina en DELIVERED y su lista de eventos es OrderConfirmed, OrderShipped y OrderDelivered, en ese orden. En cambio, ship("") sobre un pedido en DRAFT lanza InvalidOrderTransition, el estado sigue en DRAFT y la lista de eventos sigue vacía.

Conclusiones

El diagrama muestra que el ciclo de vida del pedido está controlado por el dominio: cada transición tiene sus condiciones y solo las operaciones válidas cambian el estado. Los Domain Events quedan como registro de los hechos que sí ocurrieron, porque se generan después de validar y de cambiar el estado, nunca antes.